

Makoya — The Complete Guide

Makoya

June 30, 2026

CONTENTS

Makoya — The Complete Guide

Part I — Makoya in Plain English

1. What Makoya actually is (a sticker for any website)

2. The widget = a doorbell (a clever two-part trick)

3. How our licensing works today (the concert wristband)

The honest truth (being a critic)

4. How we're tight on security (the vault that locks itself)

5. The scanner = renting a car every single time

6. Where our widget script lives today (a supply closet is better)

7. The rest of the engine, in everyday terms

8. The honest scorecard

9. The upgrade shopping list

Part II — The Technical & Business Snapshot

The stack at a glance

Can we handle 10,000 users at once?

Phases

Part III — Where We're Going (future plans)

1. Turning the widget into an MCP (giving Makoya a phone line for AI)

2. More widget features, forever, with zero customer effort

3. The infrastructure roadmap (now → next → later)

4. The bigger vision

5. One-paragraph summary for the room

Makoya — The Complete Guide

One document, three lenses. **Part I** explains the whole project in plain English with real-life comparisons (no coding knowledge needed). **Part II** is a tighter technical and business snapshot for a technical audience. **Part III** is where we're going next. Read Part I to *understand* it, Part II to *defend* it in a technical meeting, Part III to *sell the vision*.

Part I — Makoya in Plain English

1. What Makoya actually is (a sticker for any website)

Think of a website like a shop. Most shops were never built with disabled visitors in mind — the text is too small, the colours are hard to read, things move and flash, links are hard to see.

Makoya is a sticker the shop owner puts on their front door. Once it's on, every visitor gets a little floating button. Click it, and a panel opens with **18 switches**: make text bigger, increase contrast, stop animations, add a reading ruler, switch to a dyslexia-friendly font, have the page read aloud, use a big cursor, and more. It speaks **4 languages** and has **9 one-click presets** (like “Low Vision” or “Dyslexia”). The shop owner pastes **one line** onto their site and never touches it again.

Makoya is really **three products that share one engine**:

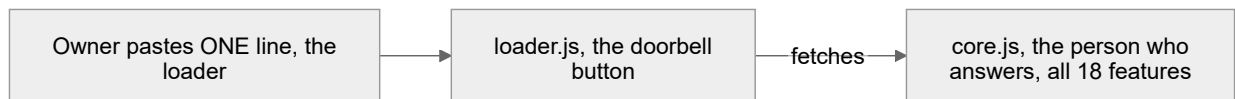
In plain words	What it does	Why it matters
The Scanner	Type in a website, get a score out of 100 and a plain-English list of what's wrong	It's both the product and the free sample that pulls people in
The Widget	The sticker itself — the floating button + 18 switches	What the shop's visitors actually use
The Dashboard	Where the owner customizes the button, sees their report, and (soon) pays	Where the business and the money live

Our goal: be the *honest* version of this. A whole industry of “accessibility overlays” got **sued** for promising “100% legally compliant” and then actually *breaking* the tools blind people rely on. We never claim “compliant.” We offer **tools and preferences**. That honesty is our edge.

2. The widget = a doorbell (a clever two-part trick)

The widget comes in **two pieces**, like a doorbell:

- **The loader** = the doorbell *button* on the wall. Tiny. Never changes. This is the one line the owner pastes.
- **The core** = the *person who answers the door*. Holds all the real features. We can swap this person for a smarter one any time — and **the button on the wall never changes**.

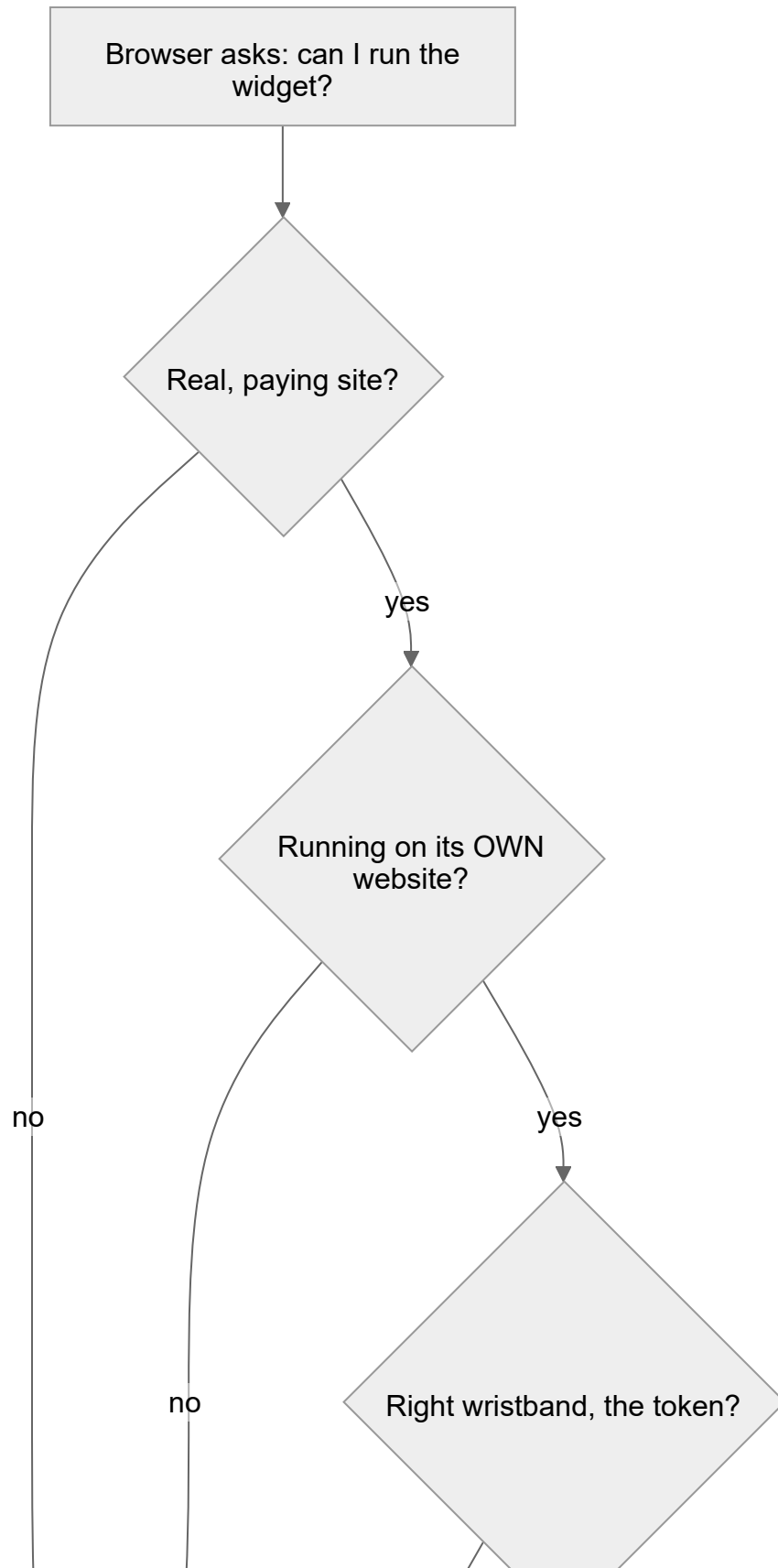


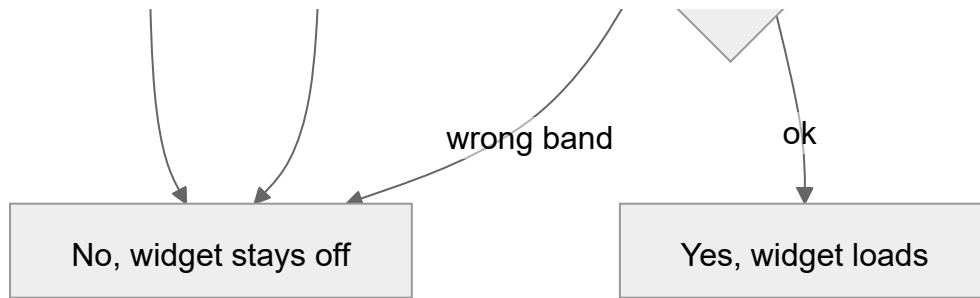
Why the pasted snippet never changes

This is why we grew from 15 to 18 features without a single customer re-pasting anything. **One rule keeps it safe:** the widget is built to **never break the host website**. If our servers are down or the internet hiccups, it quietly does nothing instead of crashing the shop's page. A broken accessibility tool is worse than none — that's exactly what got competitors sued.

3. How our licensing works today (the concert wristband)

We only want **paying customers** running the widget, and only on **their own website**. When a visitor's browser loads the widget, it phones home and asks *"is this site allowed right now?"* Our server checks three things, like a bouncer:





The bouncer's three checks

The **wristband** is a secret code we make with a one-way maths trick (an HMAC signature): we mix the site's ID with a secret only our server knows, producing a code that's easy for us to check but practically impossible to fake. We even compare it in a way that hides timing hints. That part is genuinely well done.

THE HONEST TRUTH (BEING A CRITIC)

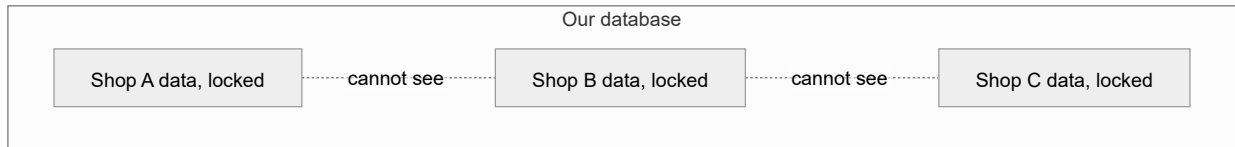
This is a **deterrent, not a vault**:

- The “is it on its own website?” check trusts a label the browser attaches (the *Origin*). A normal browser tells the truth, but a determined freeloader using special tools can **lie about it**. Like a wristband checked by someone who only glances: stops casual gate-crashers, not a pro.
- The wristband **never changes and is visible in the page source**. Someone could copy it. We deliberately decided that's fine — **who mounts a heist to steal an accessibility widget?** The reward isn't worth the effort.
- We currently run in “**watch first**” mode: the bouncer writes down who *would* be turned away but still lets them in, so we see real traffic before flipping on hard enforcement and risking locking out a real customer.

How we'd bulletproof it later (only if it matters): wristbands that **expire every few minutes** and refresh automatically, plus proving domain ownership the way Google does (drop a special file once). More moving parts — so we deliberately haven't built it. The current version is the right amount of security **for the actual risk**. Good judgement, not a gap.

4. How we're tight on security (the vault that locks itself)

Several things here are **as good as they need to be** — “improving” them would add **risk, not value**.



Each customer's data is sealed off

1. The vault locks itself (Row-Level Security). Imagine a bank where each vault opens *only* with its owner's key — and the **vault door itself** enforces it, not a guard who might forget. In most apps the code has to *remember* to filter “show this user only their own stuff,” and one forgotten line leaks everyone's data. We pushed that rule **into the database itself**, so even with a bug in our app, the database physically refuses to hand Shop B's data to Shop A. An automated test even tries to break across accounts and confirms it fails. **Rock-solid; should not change.**

2. The most sensitive list is staff-only. Sales leads live in a table that **even logged-in customers cannot read** — only our trusted server can.

3. The master key never leaves the building. The powerful database “master key” **never** reaches anyone's browser. The widget on a visitor's page can only read a tiny, harmless, public list of display settings.

4. Our scanner can't be turned into a weapon. A clever attacker might ask us to “scan” an internal/private address to snoop where they shouldn't. We refuse those. (Technical name: SSRF defense.)

5. Every door has a lock and a check. Every incoming request is validated against a strict template before we trust a character of it. Bad input is rejected with a generic error that leaks nothing.

6. Abuse protection. Strangers can only scan a few times a minute before they're slowed — nobody can hammer our scanner to run up the bill, even across all our servers at once.

The honest, mild caveats: one free security toggle on our login provider to switch on (30 seconds), and a couple of old test keys to rotate. Both listed; neither exposed in production.

5. The scanner = renting a car every single time

To truly check accessibility you can't just read code — you must **actually open the page like a real visitor**, let it load, and inspect what's on screen. So for every scan we spin up an **invisible web browser** (a real Chrome, no window), load the site, and run professional checks (the industry-standard “axe” engine plus a second engine and our own).

The catch: opening a whole invisible browser is **heavy** — like renting a car to drive to the corner shop — and we rent it **fresh every time**, inside small rented compute boxes.



Scanning today, renting a browser inside a tiny box

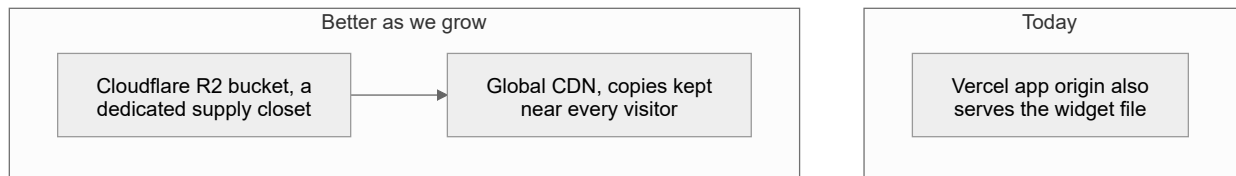
Smart thing we already did: *scheduled* re-scans don't make anyone wait — they get a **deli ticket** (a queue) and a worker handles them out back.

Honest critique: the *interactive* scan (someone clicks “scan now” and waits) still rents the car inline, and that's the **single most fragile part of our whole stack**. Sometimes the rental desk is slow to open (a *cold start* — unpacking a ~50MB browser), sometimes the box runs out of time or memory. We can only run a limited number at once because each eats lots of memory.

The upgrade (you guessed right): hand the browser problem to a company whose *entire job* is running fleets of always-warm browsers — **Browserless.io** or **Browserbase** (or Cloudflare's browser-rendering). Instead of renting and warming a car each time, we call a car service with engines always running. Costs money and adds an outside dependency, but removes our most brittle piece and lets us scan **far more sites at once**.

6. Where our widget script lives today (a supply closet is better)

Right now the widget files are served **from the same place our app runs** (Vercel). It works — but that little script downloads on **every page view of every customer's site**, potentially millions of times. Two real problems:

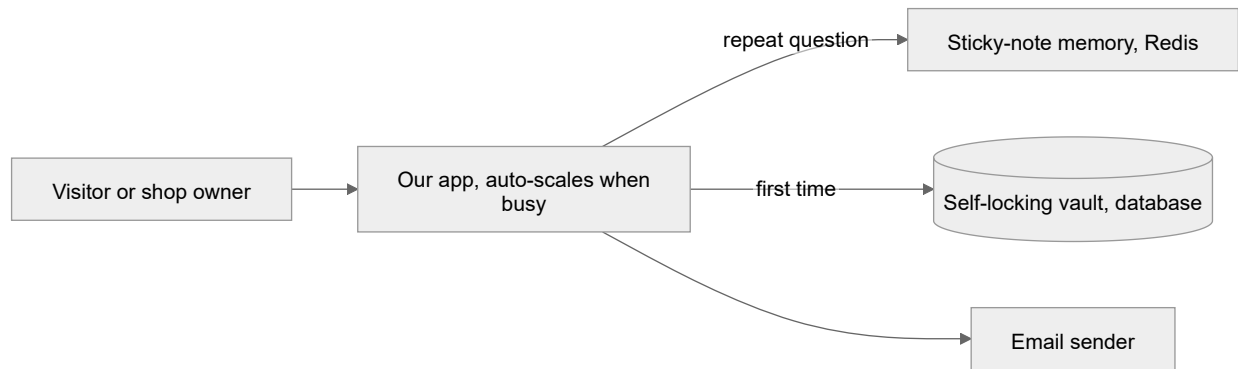


Today vs. the better setup

1. **Not as close to your visitors.** A proper *CDN* keeps copies in hundreds of cities, so a visitor in Tokyo gets it from Tokyo. Faster widget, better first impression.
2. **Cost of data leaving the building.** Most hosts charge every time a file is downloaded. **Cloudflare R2** — the bucket you mentioned — charges **zero** for that. For a file downloaded millions of times, real money.

One more honest catch: our pasted snippet points at a `...vercel.app` address. If we ever move hosts or rename the project, **every customer's pasted line breaks**. Before scaling we should put the script behind a **stable address we own forever** (like `cdn.makoya.com`) on R2 + a CDN, with version numbers on the files. A “do it before thousands of customers” item, not an emergency.

7. The rest of the engine, in everyday terms



How a normal request flows

- **It hires more help automatically.** No single computer to overwhelm; when traffic spikes, the host **adds more copies** of our app, then removes them when quiet.
- **A sticky note for repeat questions (cache).** The widget’s “am I allowed?” question fires on *every* page view. Instead of running to the vault each time, we keep the answer on a **sticky note** for a bit — making that check **3–4× faster** in our real test.
- **A deli ticket for slow jobs (queue).** Heavy background work gets a ticket and is done out back, so nobody waits at the counter.
- **Email that never blocks a sale.** If the email service hiccups, we **still save the lead**.

8. The honest scorecard

Area	How solid?	Verdict
Data isolation (self-locking vault)	● Rock-solid	Leave it. Improving adds risk, not value.
Leads kept staff-only	● Rock-solid	Leave it.
Master key never leaves server	● Rock-solid	Leave it.
Widget never breaks the host site	● Rock-solid	This is our reputation. Leave it.
Abuse / rate protection	● Solid	Good for now; revisit limits as we grow.
Licensing wristband	● Good <i>enough</i> for the risk	Honest deterrent. Upgrade to expiring tokens only if theft gets real.
Serving the widget file	● Works, not ideal	Improve: Cloudflare R2 + CDN + a domain we own.
Interactive scanner browser	● Most fragile	Improve: managed browsers (Browserless / Browserbase).
Database tier	● On free tier	Improve: paid tier + connection pooling.
Taking payments	● Not built yet	Build: wire Stripe. Plumbing is ready.

9. The upgrade shopping list

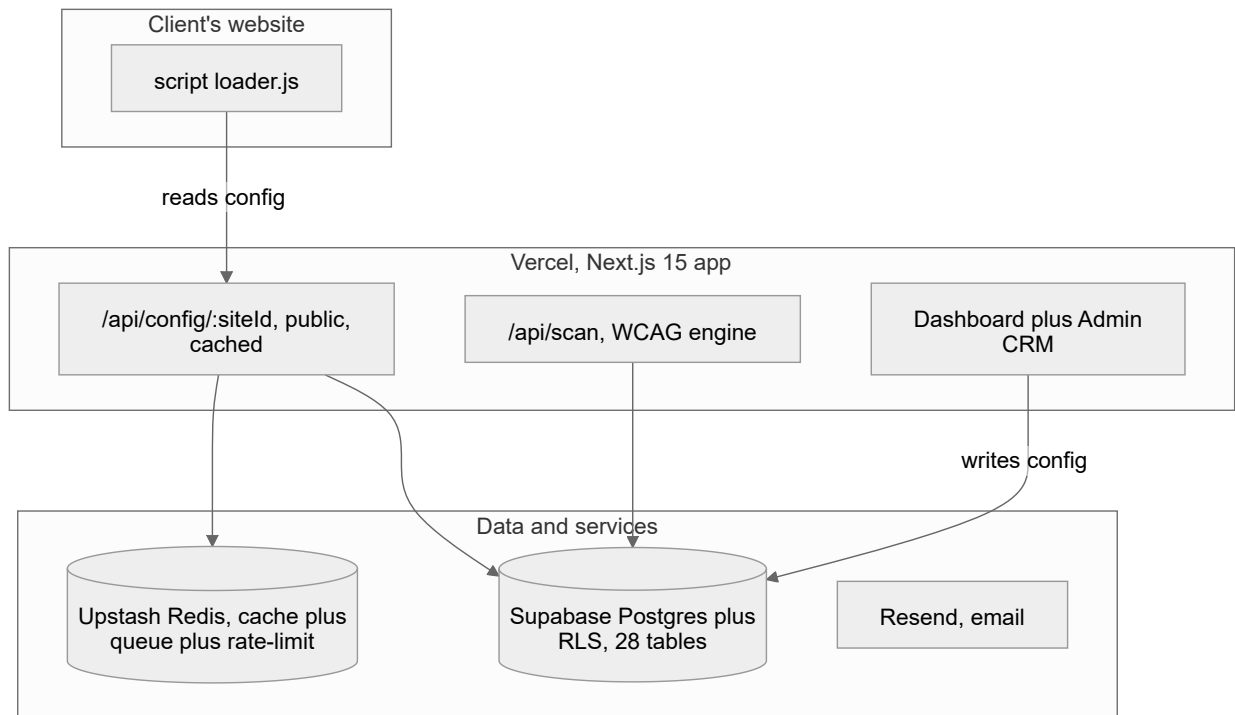
Upgrade	Roughly	What it fixes, in plain words
Stripe (payments)	free + ~3%/sale	Lets customers actually pay us. The gap between demo and business.
Supabase Pro (database)	\$25/mo	Database stops “falling asleep”; handles big bursts without choking.
Vercel Pro (hosting)	~\$20/mo	More simultaneous capacity; scanner can run every minute, not once a day.
Browserless / Browserbase	usage-based	Removes our most fragile part — no cold-start browser failures; scan many at once.
Cloudflare R2 + CDN	pennies + \$0 egress	Widget loads faster worldwide, near-free even at millions of downloads.

From “great demo” to “confidently handles serious traffic”: roughly **\$45–70/month** plus small usage fees. **No rebuild** — these are bolt-ons, because every piece sits behind a clean, swappable connector.

Part II — The Technical & Business Snapshot

For a technical reader: the same system, named properly.

The stack at a glance



System architecture

- **Monorepo (npm workspaces):** `packages/shared` (canonical config type), `packages/widget` (loader + core bundles), `apps/web` (Next.js 15 / React 19: dashboard, admin CRM, scanner, ~38 API routes, Supabase auth). CI fails if the generated config mirror drifts, so the widget and app can't disagree.
- **Tools and why:** Vercel (auto-scaling serverless), Supabase (Postgres + RLS + Auth), Upstash Redis (cache/queue/rate-limit, no-ops if unconfigured), Resend (email), Sentry (error monitoring), PostHog (product analytics), Playwright + axe-core (scanner). Each external service sits behind a **single seam** and **fails open**.

- **Database:** 28 tables, **all Row-Level-Security enabled**; `leads` and `consultation_requests` are service-role-only (RLS on, no policy). A CI job applies every migration to a throwaway Postgres and asserts cross-tenant denial.
- **Backend:** ~38 routes in four rings — public (config, scan, lead capture), authed (owner-scoped via RLS), admin (email-gated), and cron/internal (secret-gated). Service key never reaches the browser.

Can we handle 10,000 users at once?

We load-tested **production** `/api/config` with ~**13,700 requests**:

Metric	Result
Cold request (cache miss → Postgres)	850–1160 ms
Warm request (cache hit → Redis)	p50 ~270 ms / p95 ~300 ms
Cache speed-up	3–4×
Errors through 200 concurrent	0%, no 5xx

Why it scales: stateless functions auto-scale horizontally; a read-through cache keeps the hot config path off Postgres; the heavy scanner is decoupled onto a Redis queue; RLS was performance-tuned with 16 covering indexes; everything fails open.

Honest gaps before we *promise* 10k simultaneous: we proved 200 concurrent, not 10k; we're on free Supabase/Vercel tiers; Postgres needs connection pooling under burst; per-minute scanner dispatch needs Vercel Pro. **Fix = flip on paid tiers (~\$45/mo) + pooling + a staged 1k→5k→10k load test. No re-architecture.**

Phases



Roadmap status

Live today: the widget, the real scanner, the full public funnel, PDF export, Supabase auth + RLS, email, rate-limiting, error monitoring, and widget licensing enforcement. Blocked only on **Stripe** for real revenue.

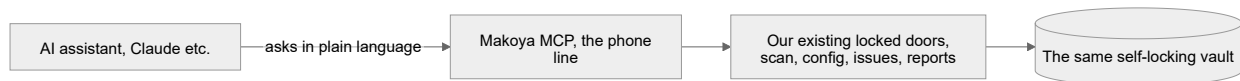
Part III — Where We're Going (future plans)

1. Turning the widget into an MCP (giving Makoya a phone line for AI)

The founder's direction is to make Makoya speak **MCP** — the new standard that lets AI assistants (like Claude) *use tools* on your behalf.

The plain-English version: today a human logs into the dashboard and clicks buttons. With MCP, you could just **tell an AI assistant:** *"scan my client's site, tell me the top 3 problems, and turn on the dyslexia profile for them"* — and the assistant does it by calling Makoya's tools directly. No clicking.

Real-life comparison: it's exactly how your bank added "Alexa, what's my balance?" The bank didn't rebuild itself — it exposed a **safe, locked phone line** to its existing system. We're doing the same.



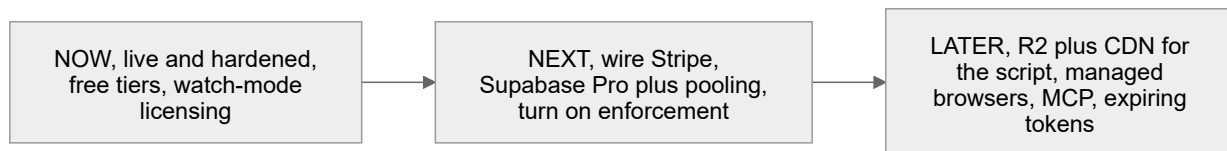
Widget-as-MCP, a thin phone line on top of what exists

Why this is realistic for us and not a fantasy: our features are *already* built as clean, separate, **locked doors** (authenticated APIs) — scan a site, read issues, change a setting, generate a report. The MCP is mostly a **thin translator** that turns “an AI’s request” into “knock on the right door.” It is **wiring, not a rebuild** — which is only true because we built tidily from day one. (If our code were a tangled mess, this would be impossible. It isn’t.)

2. More widget features, forever, with zero customer effort

Remember the doorbell: the **button on the wall (loader)** never changes, we just upgrade the **person who answers (core)**. So we can keep adding accessibility tools — voice control, smarter read-aloud, AI-suggested fixes — and **every existing customer gets them automatically** without touching their site. We already proved this going 15 → 18 features.

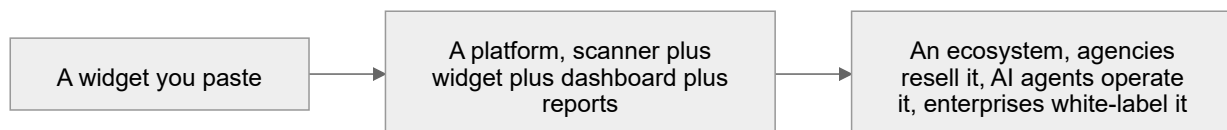
3. The infrastructure roadmap (now → next → later)



The upgrade timeline

- **Next (turn it into a business):** wire **Stripe** so people can pay; move to **Supabase Pro** + connection pooling so the database never sleeps and survives bursts; flip licensing from “watch” to “enforce.”
- **Later (scale and polish):** move the widget script to **Cloudflare R2 + CDN** behind a domain we own (faster + near-free); hand scanning to a **managed browser service** (kills our most fragile piece); ship the **MCP phone line**; add **expiring licensing tokens** if widget theft ever becomes a real problem.

4. The bigger vision



From tool to platform

We started as a **sticker you paste on a site**. We’re already a **platform** (scan

- widget + dashboard + compliance reports). The future is an **ecosystem**: agencies resell it to their clients (the partner system is already in the database), AI agents operate it through MCP, and enterprises run it under their own brand (white-label, also already scaffolded). The same self-locking vault and clean locked doors carry us through all three stages **without a rewrite** — that's the whole point of building it carefully now.

5. One-paragraph summary for the room

Makoya is a paste-once accessibility widget plus a real scanner plus an owner dashboard. What we're proud of — a database that locks itself so no customer can see another's data, a widget engineered never to break the host page, and abuse protection — is rock-solid and shouldn't be touched. What we're honest about: our licensing is a smart deterrent rather than an un-pickable lock, the scanner's invisible browser is our most fragile piece and should move to a managed service, and the widget file should move to a cheap, fast, global bucket. None of it needs a rebuild — it's a \$45–70/month upgrade path. The one thing between us and revenue is wiring Stripe, and that's already plumbed. And because every feature is already a clean locked door, our future — letting AI assistants operate Makoya through MCP, reselling through agencies, and white-labelling for enterprise — is wiring on top of what exists, not starting over.